



Laboratório 4: Autenticação de usuários com senha

23/10/2025

1 Sistema de autenticação de usuários

Neste laboratório, iremos implementar um sistema de autenticação de usuários em Java. Apesar da simplicidade do sistema, serão vistas as boas práticas de segurança na implementação de autenticação de usuários, como o uso de algoritmos de *hash* seguros e a separação das camadas de persistência. Os códigos apresentados na Seção 2 devem ser utilizados como base para a implementação do referido sistema.

1.1 Requisitos funcionais

Requisitos funcionais são as funcionalidades que o sistema deve apresentar, ou seja, aquilo que o cliente espera que o sistema faça. São descritos em termos de entradas, processamento e saídas do sistema. Os requisitos funcionais do sistema de autenticação de usuários são:

RF1: O sistema deve permitir o cadastro de novos usuários, com login e senha;

RF2: O sistema deve permitir a atualização da senha de um usuário;

RF3: O sistema deve permitir a autenticação de um usuário, ou seja, verificar se o login e a senha informados estão corretos.

1.2 Requisitos arquiteturais

Os requisitos arquiteturais são um tipo de requisito não funcional que influenciam o projeto da arquitetura do sistema. Definem restrições ou qualidades que o sistema deve ter para garantir que ele seja, por exemplo, fácil de manter, escalável, seguro, eficiente, entre outros. Os requisitos arquiteturais do sistema de autenticação de usuários são:

RA1: O sistema deve ter uma classe `User` com os atributos `login` e `password`.

RA2: O sistema deve ser implementado com o padrão de projeto *Service*¹, que separa a lógica de negócio da lógica de apresentação (interface com o usuário).

- A classe `UserService` deve ter os seguintes métodos:
 - `register`: para cadastrar um novo usuário. Caso o login já exista, retornar uma exceção personalizada (Ex: `UserAlreadyExistsException`);
 - `updatePassword`: para atualizar a senha de um usuário. Caso o usuário não exista, retornar uma exceção personalizada (Ex: `UserNotFoundException`). Caso a senha informada não corresponda a atual senha do usuário, deve retornar uma exceção personalizada (Ex: `InvalidPasswordException`);
 - `authenticate`: para simular a autenticação de um usuário. Caso o login ou a senha informados estejam incorretos, deve retornar uma exceção personalizada (Ex: `InvalidLoginException`).

¹<https://java-design-patterns.com/patterns/service-layer/>

- O algoritmo de *hash* de senha (e.g. PBKDF2 ou BCrypt) deve ser configurável, ou seja, deve ser possível alterar o algoritmo de *hash* sem alterar o código da aplicação. Assim, ao instanciar a classe `UserService`, deve-se passar o algoritmo de *hash* a ser utilizado. O algoritmo BCrypt deve ser o padrão, mas o usuário deve poder escolher outro algoritmo de *hash* ao iniciar a aplicação.

RA3: O sistema deve ser implementado com o padrão de projeto *Repository*², que separa a lógica de acesso a dados da lógica de negócio.

- A interface `UserRepository` deve ter os seguintes métodos:
 - `save`: recebe um objeto do tipo `User` e o armazena na camada de persistência;
 - `update`: recebe um objeto do tipo `User` e atualiza os dados do usuário na camada de persistência;
 - `findByLogin`: recebe um login e retornar o objeto do tipo `User` correspondente ao login informado, ou `null` caso não exista um usuário com o login informado.
- A interface `UserRepository` consiste em uma abstração da camada de persistência, ou seja, independente da tecnologia utilizada para armazenar os dados (Ex: em memória, em arquivos de texto ou em um banco de dados relacional).
 - Você deverá prover uma implementação para armazenar os dados dos usuários em uma coleção em memória (Ex: `List<User>` ou `Map<String, User>`). Mas se desejar, você pode prover outras implementações para armazenar os dados dos usuários em arquivos de texto ou em um banco de dados relacional (e.g. H2, SQLite, MySQL, PostgreSQL, etc.).

Nota

O método `System.console().readPassword()` pode ser utilizado para ler a senha do usuário sem exibi-la na tela. O método `readPassword` retorna um `char[]` com a senha informada pelo usuário. Porém, se você estiver executando a aplicação a partir de uma IDE ou com `gradle run`, não será possível obter um console. Para usar esse método, execute a aplicação da seguinte forma:

```
# Para gerar os scripts de execução. Se estiver no windows, use gradlew.bat
./gradlew installDist

# Para executar a aplicação com o script gerado. Se estiver no windows, use app.bat
./app/build/install/app/bin/app
```

2 Exemplo como usar algoritmos para fazer *hash* de senhas

Nesta seção vamos apresentar um exemplo de como usar algoritmos de *hash* para fazer o *hash* de senhas e demonstrar o quão lento é o algoritmo BCrypt em comparação com os algoritmos MD5 e SHA-256. Para isso, vamos usar a biblioteca `javax.crypto`, que faz parte do JDK, e o algoritmo BCrypt que será provido pelo *framework Spring Security*³.

- Crie um projeto Java com gradle e no arquivo de configuração `build.gradle.kts` faça as inclusões conforme apresentado na Listagem 1.
 - Inclua as dependências necessárias para usar o *framework Spring Security* e o algoritmo BCrypt

²<https://java-design-patterns.com/patterns/repository/>

³<https://spring.io/projects/spring-security>

- Adicione os plugins `io.spring.dependency-management` e `org.springframework.boot`

Listagem 1: Inclusões no arquivo `build.gradle.kts`

```
plugins {  
    application  
    // Adicione os plugins abaixo  
    id("org.springframework.boot") version "3.4.5"  
    id("io.spring.dependency-management") version "1.1.7"  
}  
  
dependencies {  
    // Framework Spring Security  
    implementation("org.springframework.boot:spring-boot-starter-security")  
    testImplementation("org.springframework.boot:spring-boot-starter-test")  
    testImplementation("org.springframework.security:spring-security-test")  
  
    // demais dependências  
}
```

Nesse projeto Java criaremos duas classes:

- A classe `PasswordHashing.java` (veja Listagem 2) contém métodos para gerar um *hash* de senha com *salt* e verificar se uma senha corresponde a um *hash* de senha.
 - São apresentados diferentes métodos para gerar o *hash* de senha, utilizando o algoritmos MD5, SHA-256, BCrypt e PBKDF2.
- A classe `App.java` (veja Listagem 3) ilustra como usar a classe `PasswordHashing.java`.
 - A aplicação demonstra o tempo necessário para gerar o *hash* de senha com diferentes algoritmos (MD5, SHA-*, BCrypt e PBKDF2). Quanto mais lento for um algoritmo de *hash*, mais seguro ele é contra ataques de força bruta.

Listagem 2: Classe `PasswordHashing.java`

```
package ads.seg;  
  
import java.nio.ByteBuffer;  
import java.nio.CharBuffer;  
import java.nio.charset.CharacterCodingException;  
import java.nio.charset.Charset;  
import java.nio.charset.CharsetEncoder;  
import java.nio.charset.StandardCharsets;  
import java.security.MessageDigest;  
import java.security.NoSuchAlgorithmException;  
import java.security.spec.InvalidKeySpecException;  
import java.security.spec.KeySpec;  
import java.util.Base64;  
  
import javax.crypto.SecretKeyFactory;  
import javax.crypto.spec.PBEKeySpec;  
  
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;  
  
/**  
 * Classe utilitária para realizar o hash de senhas com diferentes algoritmos.  
 *  
 * Algoritmos de hash com message digest: MD5, SHA-1, SHA-256, SHA-512  
 */
```

```

* Algoritmos de hash com PBKDF2: PBKDF2WithHmacSHA1, PBKDF2WithHmacSHA256, PBKDF2WithHmacSHA512
* BCrypt é um algoritmo de hash de senhas
*
*/
public abstract class PasswordHashing {

    // Charset padrão para codificação de strings
    private static final Charset charset = StandardCharsets.UTF_8;

    /**
     * Método que recebe uma senha e a converte para um array de bytes
     * @param password senha do usuário em formato char[]
     * @return array de bytes da senha
     * @throws CharacterCodingException caso ocorra um erro de codificação
     */
    private static byte[] convertCharToByte(char[] password) {
        byte[] bytePassword = new byte[0];
        try {
            CharsetEncoder encoder = PasswordHashing.charset.newEncoder();
            CharBuffer charBuffer = CharBuffer.wrap(password);
            ByteBuffer byteBuffer = encoder.encode(charBuffer);
            bytePassword = new byte[byteBuffer.remaining()];
            byteBuffer.get(bytePassword);
        } catch (CharacterCodingException e) {
            throw new RuntimeException("Erro ao codificar", e);
        }
        return bytePassword;
    }

    /**
     * Método que recebe uma senha, um salt e um algoritmo de hash e retorna o hash da senha
     * @param password senha do usuário
     * @param salt salt aleatório
     * @param algorithm algoritmo de hash
     * @return hash da senha
     * @throws NoSuchAlgorithmException caso o algoritmo de hash não exista
     */
    public static byte[] hashPasswordWithMessageDigest(char[] password, byte[] salt, String
algorithm) throws NoSuchAlgorithmException {
        MessageDigest md = MessageDigest.getInstance(algorithm);
        md.update(salt);
        return md.digest(convertCharToByte(password));
    }

    /**
     * Método que verifica se a senha fornecida é igual ao hash da senha armazenado
     * @param password senha do usuário
     * @param salt salt aleatório
     * @param algorithm algoritmo de hash
     * @param hashedPassword hash da senha armazenado
     * @return true se a senha fornecida é igual ao hash da senha armazenado, false caso contrário
     * @throws NoSuchAlgorithmException caso o algoritmo de hash não exista
     */
    public static boolean verifyPasswordWithMessageDigest(char[] password, byte[] salt, String
algorithm, byte[] hashedPassword) throws NoSuchAlgorithmException {
        return MessageDigest.isEqual(hashPasswordWithMessageDigest(password, salt, algorithm),
hashedPassword);
    }

    /**
     * Método que recebe uma senha, um salt, um algoritmo de hash, número de iterações e tamanho
da chave e retorna o hash da senha
     * @param password senha do usuário

```

```

    * @param salt salt aleatório
    * @param algorithm algoritmo de hash
    * @param iterations número de iterações
    * @param keyLength tamanho da chave
    * @return hash da senha
    * @throws NoSuchAlgorithmException caso o algoritmo de hash não exista
    * @throws InvalidKeySpecException caso a chave seja inválida
    */
    public static byte[] hashPasswordWithPBKDF2(char[] password, byte[] salt, String algorithm,
        int iterations, int keyLength) throws NoSuchAlgorithmException, InvalidKeySpecException {

        KeySpec spec = new PBEKeySpec(password, salt, iterations, keyLength);
        return SecretKeyFactory.getInstance(algorithm).generateSecret(spec).getEncoded();
    }

    /**
     * Método que verifica se a senha fornecida é igual ao hash da senha armazenado
     * @param password senha do usuário
     * @param salt salt aleatório
     * @param algorithm algoritmo de hash
     * @param iterations número de iterações
     * @param keyLength tamanho da chave
     * @param hashedPassword hash da senha armazenado
     * @return true se a senha fornecida é igual ao hash da senha armazenado, false caso contrário
     * @throws NoSuchAlgorithmException caso o algoritmo de hash não exista
     * @throws InvalidKeySpecException caso a chave seja inválida
     */
    public static boolean verifyPasswordWithPBKDF2(char[] password, byte[] salt, String algorithm,
        int iterations, int keyLength, byte[] hashedPassword) throws NoSuchAlgorithmException,
        InvalidKeySpecException {
        Base64.Encoder encoder = Base64.getEncoder();
        return encoder.encodeToString(hashPasswordWithPBKDF2(password, salt, algorithm, iterations
            , keyLength)).equals(encoder.encodeToString(hashedPassword));
    }

    /**
     * Método que recebe uma senha e retorna o hash da senha com BCrypt
     *
     * Não é necessário fornecer um salt, pois o BCrypt já gera um salt aleatório internamente
     * @param password senha do usuário
     * @param salt salt aleatório
     * @return hash da senha
     */
    public static byte[] hashPasswordWithBCrypt(char[] password) {
        BCryptPasswordEncoder encoder = new BCryptPasswordEncoder();
        return encoder.encode(CharBuffer.wrap(password)).getBytes(PasswordHashing.charset);
    }

    /**
     * Método que verifica se a senha fornecida é igual ao hash da senha armazenado com BCrypt
     *
     * Não é necessário fornecer um salt, pois o BCrypt já gera um salt aleatório internamente
     * @param password senha do usuário
     * @param salt salt aleatório
     * @param hashedPassword hash da senha armazenado
     * @return true se a senha fornecida é igual ao hash da senha armazenado, false caso contrário
     */
    public static boolean verifyPasswordWithBCrypt(char[] password, byte[] hashedPassword) {
        BCryptPasswordEncoder encoder = new BCryptPasswordEncoder();
        return encoder.matches(CharBuffer.wrap(password), new String(hashedPassword,
            PasswordHashing.charset));
    }
}

```

```
}
```

Listagem 3: Classe App.java

```
package ads.seg;

import java.io.Console;
import java.io.IOException;
import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;
import java.security.spec.InvalidKeySpecException;
import java.time.Duration;
import java.util.Base64;
import java.util.Scanner;

import javax.swing.JOptionPane;
import javax.swing.JPasswordField;
import javax.swing.SwingUtilities;

/**
 * Classe principal para testar a classe PasswordHashing com diferentes algoritmos de hash.
 */
public class App {
    private static final SecureRandom S_RANDOM = new SecureRandom();

    public static void main(String[] args) throws IOException {

        Duration start, end;
        String msg = "Hashed password with %s in %d nanoseconds.\n(base64): %s\n";
        Base64.Encoder encoder = Base64.getEncoder();

        byte[] salt = new byte[16];
        S_RANDOM.nextBytes(salt);
        String[] algorithms = { "MD5", "SHA-1", "SHA-256", "SHA-512" };
        byte[] hashedPassword;

        // Evite usar String para senhas, use char[] ou byte[]. A String é imutável e pode ser
        // mantida na memória por mais tempo do que o necessário, tornando-a vulnerável a ataques
        // de memória.
        // Use char[] ou byte[] para senhas, pois eles podem ser limpos da memória após o uso.
        // Isso reduz a janela de oportunidade para um invasor acessar a senha.
        // java.util.Arrays.fill(password, '0'); // Limpa a senha após o uso
        char[] password = {'1', '2', '3', '4', '5', '6'}; // Senha a ser testada

        // Testando algoritmos de hash com message digest MD5, SHA-1, SHA-256, SHA-512 e
        // comparando o tempo de execução
        for (String algorithm : algorithms) {
            try {
                start = Duration.ofNanos(System.nanoTime());
                hashedPassword = PasswordHashing.hashPasswordWithMessageDigest(password, salt,
algorithm);
                end = Duration.ofNanos(System.nanoTime());
                System.out.println(String.format(msg, algorithm, end.minus(start).toNanos(),
encoder.encodeToString(hashedPassword)));
            } catch (NoSuchAlgorithmException e) {
                e.printStackTrace();
            }
        }

        int iterations = 210000; // Número de iterações
        int keyLength = 128; // Tamanho da chave
    }
}
```

```

String algorithm = "PBKDF2WithHmacSHA512";
// Testando algoritmos de hash com PBKDF2 e comparando o tempo de execução
try {
    start = Duration.ofNanos(System.nanoTime());
    hashedPassword = PasswordHashing.hashPasswordWithPBKDF2(password, salt, algorithm,
iterations, keyLength);
    end = Duration.ofNanos(System.nanoTime());
    System.out.println(String.format(msg, algorithm, end.minus(start).toNanos(), encoder.
encodeToString(hashedPassword)));
} catch (NoSuchAlgorithmException | InvalidKeySpecException e) {
    e.printStackTrace();
}

// Testando algoritmo de hash BCrypt e comparando o tempo de execução
algorithm = "BCrypt";
start = Duration.ofNanos(System.nanoTime());
hashedPassword = PasswordHashing.hashPasswordWithBCrypt(password);
end = Duration.ofNanos(System.nanoTime());
System.out.println(String.format(msg, algorithm, end.minus(start).toNanos(), encoder.
encodeToString(hashedPassword)));

// Isso não funcionará em IDEs como IntelliJ IDEA ou se executar com o gradle run,
// pois o console não está disponível. Use o terminal ou o console do sistema operacional
para testar.
// char[] s = System.console().readPassword("Entre com a senha: ");
// System.out.println("Senha: " + new String(s));
}
}

```

- A classe `PasswordHashingTest.java` (veja Listagem 4) contém testes de unidade para a classe `PasswordHashing.java`. Os testes verificam se a senha gerada com um algoritmo de *hash* é igual ao *hash* gerado com a mesma senha.
 - Os teste de unidade são executados com o *framework JUnit*⁴ e essa classe deve estar dentro do diretório `src/test/java`.

Listagem 4: Classe `PasswordHashingTest` com testes de unidade

```

package ads.seg;

import static org.junit.jupiter.api.Assertions.assertTrue;

import java.security.NoSuchAlgorithmException;
import java.security.SecureRandom;
import java.security.spec.InvalidKeySpecException;

import org.junit.jupiter.api.Test;

public class PasswordHashingTest {

    private static final SecureRandom S_RANDOM = new SecureRandom();

    @Test
    public void testHashPasswordWithMessageDigest() {
        byte[] salt = new byte[16];
        S_RANDOM.nextBytes(salt);
        String[] algorithms = { "MD5", "SHA-1", "SHA-256", "SHA-512" };
        byte[] hashedPassword;
    }
}

```

⁴<https://junit.org/junit5/>

```

// Evite usar String para senhas, use char[] ou byte[]. A String é imutável e pode ser
// mantida na memória por mais tempo do que o necessário, tornando-a vulnerável a ataques
// de memória.
// Use char[] ou byte[] para senhas, pois eles podem ser limpos da memória após o uso.
// Isso reduz a janela de oportunidade para um invasor acessar a senha.
// java.util.Arrays.fill(password, '0'); // Limpa a senha após o uso
char[] password = {'1', '2', '3', '4', '5', '6'}; // Senha a ser testada

for (String algorithm : algorithms) {
    try {
        hashedPassword = PasswordHashing.hashPasswordWithMessageDigest(password, salt,
algorithm);
        assertTrue(PasswordHashing.verifyPasswordWithMessageDigest(password, salt,
algorithm, hashedPassword));
    } catch (NoSuchAlgorithmException e) {
        e.printStackTrace();
    }
}

@Test
public void testHashPasswordWithPBKDF2() {
    byte[] salt = new byte[16];
    S_RANDOM.nextBytes(salt);
    int iterations = 600000;
    int keyLength = 128;
    String[] algorithms = { "PBKDF2WithHmacSHA1", "PBKDF2WithHmacSHA256", "
PBKDF2WithHmacSHA512" };
    byte[] hashedPassword;
    char[] password = {'1', '2', '3', '4', '5', '6'}; // Senha a ser testada
    for (String algorithm : algorithms) {
        try {
            hashedPassword = PasswordHashing.hashPasswordWithPBKDF2(password, salt, algorithm,
iterations,
                keyLength);
            assertTrue(PasswordHashing.verifyPasswordWithPBKDF2(password, salt, algorithm,
iterations, keyLength,
                hashedPassword));
        } catch (NoSuchAlgorithmException | InvalidKeySpecException e) {
            e.printStackTrace();
        }
    }
}

@Test
public void testHashPasswordWithBCrypt() {
    char[] password = {'1', '2', '3', '4', '5', '6'}; // Senha a ser testada
    byte[] hashedPassword = PasswordHashing.hashPasswordWithBCrypt(password);
    assertTrue(PasswordHashing.verifyPasswordWithBCrypt(password, hashedPassword));
}
}

```